DOFBOT User Manual

A step-by-step manual for the Yahboom DOFBOT ROS 2 project: from a blank machine to a robot arm that plans its own motions, is driven from the keyboard, and picks up objects it sees with a camera.

Written for students with little or no ROS experience. Follow the sections in order the first time. Every section is numbered so you can be told "go back to §6.3" and know exactly where to look.

Table of contents

1. Before you start

1.1 What you are building

A 5-joint robot arm with a gripper that you can control four different ways, each one a step up from the last:

1. **By hand, in simulation.** Drag a marker in a 3D viewer and the virtual arm follows (§6).
2. **By keyboard.** Press a key and the real arm nudges (§9).
3. **By plan.** You give a goal, software works out the path, and the arm executes it (§10).
4. **By itself.** A camera sees an object and the arm goes and picks it up (§11), or plays a game of chess against you (§12).

1.2 What you need

Software (minimum, for simulation only)

- A computer with Ubuntu 22.04, or Windows 11 with WSL2 + Docker
- About 15 GB of free disk space
- No robot needed for §5, §6, §7 and §10

Hardware (to run anything physical)

- A Yahboom DOFBOT arm with its **DC power supply**. USB power alone cannot drive the servos, so this is not optional
- The USB serial cable (a CH340 adapter, USB vendor:product `1a86:7523`)
- Optionally a Raspberry Pi 4 (§3) if you want the arm controlled by a small computer instead of your laptop
- Optionally the USB camera, for §11

1.3 The two ways this project can be wired

Setup A: everything on one machine. Your laptop runs everything and the arm plugs into your laptop's USB. Simplest. This is the setup the project was built and tested on.

```
[ your laptop ]  --USB serial-->  [ DOFBOT arm ]
  driver + MoveIt + RViz + tools
```

Setup B: split across a Pi. The Raspberry Pi sits next to the arm and runs the driver. Your laptop runs the heavy graphical stuff and talks to the Pi over WiFi. This is §3 and §4.

```
[ your laptop ]  --WiFi/ROS-->  [ Raspberry Pi ]  --USB serial-->  [ DOFBOT arm ]
  MoveIt + RViz                    driver
```

Which should you pick? Start with Setup A. Get the arm moving from one machine first. Only move to Setup B once §8 works, because if something breaks in Setup B you will not know whether the problem is the robot or the network.

1.4 The other documents in this repo

Document	What it is for
this manual	the full course, read it in order the first time
`docs/demo_runbook.md`	cold-machine checklist for the chess demo (§12)

1.5 Safety rules, read once and follow always

1. **Pose the arm roughly straight up before starting the driver.** The driver has no way to read where the servos actually are (§7.10). It assumes the arm starts centered. If it starts folded over, the first command will make it snap upright, fast.
2. **Keep the workspace clear.** There is no obstacle model. Nothing in the software knows your desk, your coffee, or your hand exists. A planned path will go straight through all three.
3. **First motions at 10 to 20% speed.** In RViz there is a Velocity Scaling box. Use it.
4. **Keep a hand on the power switch.** It is the only emergency stop this robot has.

2. Installing ROS 2 and the dependencies

2.1 Choosing your path

Your machine	Follow
Ubuntu 22.04 (native or a VM)	§2.2
Windows 11 with WSL2	§2.3
macOS, or other Linux	§2.4 (simulation only)

The project targets **ROS 2 Humble**, which officially pairs with Ubuntu 22.04. Do not substitute another Ubuntu version. The ROS packages simply do not exist for it, and mixing versions is the most common way students lose a day.

2.2 Path A: Ubuntu 22.04

2.2.1 Add the ROS 2 software source

Ubuntu does not ship ROS. You are telling apt about an extra repository.

```
sudo apt update && sudo apt install -y software-properties-common curl
sudo add-apt-repository universe

sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
  -o /usr/share/keyrings/ros-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" \
  | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

sudo apt update
```

The key file proves the packages really come from the ROS project. If the `curl` line fails, check the current instructions at docs.ros.org. This is the one step that occasionally changes.

2.2.2 Install ROS 2 and the project's dependencies

```
sudo apt install -y \
  ros-humble-desktop \
  ros-humble-moveit \
  ros-humble-ros2-control ros-humble-ros2-controllers \
  ros-humble-v4l2-camera ros-humble-rqt-image-view \
  python3-colcon-common-extensions python3-serial python3-yaml \
  v4l-utils git stockfish
```

What each piece is for:

Package	Why this project needs it
`ros-humble-desktop`	ROS 2 itself, plus RViz (the 3D viewer)
`ros-humble-moveit`	motion planning, works out how to get from A to B

Package	Why this project needs it
`ros2-control` / `ros2-controllers`	only used by the **simulation** (§6.3)
`v4l2-camera`, `v4l-utils`	USB camera input (§11)
`rqt-image-view`	a window to look at camera images
`colcon-common-extensions`	the build tool (§5.5)
`python3-serial`	lets the driver write bytes to the USB port
`stockfish`	the chess engine (§12)

2.2.3 Let your user access the USB serial port

```
sudo usermod -aG dialout $USER
```

Log out and back in for this to take effect. Without it you get `Permission denied: /dev/ttyUSB0`.

2.2.4 Make every new terminal know about ROS

```
echo 'source /opt/ros/humble/setup.bash' >> ~/.bashrc
```

§5.3 explains what "source" means. For now: without it, the `ros2` command does not exist.

2.2.5 Get the project

```
mkdir -p ~/ros2_ws && cd ~/ros2_ws
git clone <this-repo-url> yahboom-robot-arm
```

Now go to §2.5.

2.3 Path B: Windows 11 + WSL2 + Docker

This is what the project was developed on. Windows runs the desktop, WSL2 runs a real Linux kernel inside Windows, and Docker runs a clean Ubuntu 22.04 container inside that. Three layers sounds heavy, but it means your Windows machine stays untouched and the ROS install can never half-break.

2.3.1 Install WSL2 and Docker Desktop

1. Install WSL2 with Ubuntu: in PowerShell, `wsl --install -d Ubuntu-22.04`.
2. Install **Docker Desktop** for Windows.
3. Docker Desktop → **Settings** → **Resources** → **WSL integration** → enable it for your Ubuntu distro → **Apply & Restart**.
4. Check inside WSL: `docker version` should print something.

2.3.2 Clone the project and create the container

From a WSL terminal:

```
cd ~ && mkdir -p ros2_ws && cd ros2_ws
git clone <this-repo-url> yahboom-robot-arm

docker pull osrf/ros:humble-desktop-full

docker run -it --net=host --name dofbot \
  -e DISPLAY=$DISPLAY -e WAYLAND_DISPLAY=$WAYLAND_DISPLAY \
  -e XDG_RUNTIME_DIR=$XDG_RUNTIME_DIR \
  --mount type=bind,source=/tmp/.X11-unix,target=/tmp/.X11-unix \
  --mount type=bind,source=/mnt/wslg,target=/mnt/wslg \
  --mount type=bind,source=$HOME/ros2_ws/yahboom-robot-arm,target=/root/yahboom-robot-arm \
  --mount type=bind,source=/dev,target=/dev \
  --device-cgroup-rule='c 188:* rmw' \
  --device-cgroup-rule='c 166:* rmw' \
  --device-cgroup-rule='c 81:* rmw' \
  osrf/ros:humble-desktop-full
```

Those unusual flags, explained:

Flag	What it does
`--net=host`	the container shares WSL's network, so ROS nodes inside and outside can see each other
`-e DISPLAY` + the two X11/WSLg mounts	lets graphical apps (RViz) draw a window on your Windows desktop
the repo `--mount`	your code lives on Windows/WSL and appears inside the container at `/root/yahboom-robot-arm`. Edit in your normal editor, build in the container. Delete the container and your code is untouched
`--mount .../dev`	USB devices plugged in **later** appear inside the container immediately
`--device-cgroup-rule`	permission to actually open those USB devices (188 = USB serial, 81 = video, 166 = modem-class)

Day to day you do not repeat that command. You use:

```
docker start dofbot            # boot the container
docker exec -it dofbot bash    # open a shell in it (repeat per terminal)
```

Or from the repo, `scripts/container.sh` does the start for you.

2.3.3 Install the dependencies inside the container

```
docker exec -it dofbot bash

apt update && apt install -y \
  ros-humble-moveit ros-humble-ros2-control ros-humble-ros2-controllers \
  python3-serial ros-humble-v4l2-camera ros-humble-rqt-image-view \
  v4l-utils stockfish
pip3 install chess
```

2.3.4 Pin the Python packages, do not skip this

```
printf 'numpy<2\nopencv-python<4.11\nsetuptools<60\n' > /root/pip-constraints.txt
echo 'export PIP_CONSTRAINT=/root/pip-constraints.txt' >> /root/.bashrc
export PIP_CONSTRAINT=/root/pip-constraints.txt
```

Why this matters. ROS Humble was built against numpy 1.x and an older setuptools. If any pip install quietly upgrades numpy to 2.x, parts of ROS stop working with errors that look nothing like the cause:

Error you will see	Real cause
`_ARRAY_API not found`	numpy 2.x got installed
`KeyError: 16`	opencv-python too new
`canonicalize_version()` TypeError	setuptools ≥ 60

The constraints file makes those upgrades impossible. Set it **before** you install anything with pip.

2.3.5 Set up the shell helpers

```
/root/yahboom-robot-arm/scripts/one_time/setup_container.sh
```

This is idempotent (safe to re-run). It makes every new shell source ROS and the workspace automatically, and defines a `ros-build` command that always builds in the right directory. Then open a fresh shell.

2.3.6 Save your work

From **WSL, not the container**:

```
docker commit dofbot dofbot:setup
```

Now the whole configured environment is a saved image. If you ever wreck the container, you can recreate it from this snapshot instead of redoing §2.3.3 to §2.3.5.

2.3.7 What this puts on your Windows machine

The complete Windows-side footprint is four items, all reversible:

Change	Purpose	Undo
Docker Desktop WSL-integration toggle	the `docker` command inside WSL	flip it off
usbipd-win, plus a `bind` record per device	USB serial into WSL (§8.2)	`usbipd unbind --all`, uninstall
ffmpeg (via winget)	the camera bridge (§11.2.2)	`winget uninstall ffmpeg`
a firewall allow-rule for ffmpeg	lets WSL reach the video stream on port 8090	Windows Security → Allow an app through firewall

No driver is permanently replaced: while a device is *attached* to WSL it disappears from Windows, and comes back on detach or unplug.

2.4 Path C: macOS or other Linux, via Docker

Use the §2.3.2 command minus the WSLg-specific mounts, and provide display access your own way. On Linux, mount `/tmp/.X11-unix` and run `xhost +local:`. On macOS, use XQuartz or skip RViz entirely.

Hard limit: Docker on macOS and on Windows-without-WSL cannot pass USB devices through at all. You can do the simulation and code sections, §5, §6, §7 and §10, but not real hardware. For real hardware you need a Linux host or the Raspberry Pi of §3.

2.5 Install the vision dependencies (only needed for §11)

YOLO and PyTorch are about 2 GB. Skip this until you reach §11.

In the container (Path B):

```
pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cpu
pip3 install ultralytics "numpy==1.26.4" "opencv-python==4.10.0.84"
```

On native Ubuntu (Path A), use a virtual environment that can still see the system ROS packages:

```
source /opt/ros/humble/setup.bash
python3 -m venv --system-site-packages ~/venvs/ros2_yolo
source ~/venvs/ros2_yolo/bin/activate
pip install -U pip
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
pip install ultralytics "numpy<2" "opencv-python<5"
```

`--system-site-packages` is the important flag: without it the venv cannot see `rclpy` and nothing ROS-related will import.

2.6 Check the install worked

```
source /opt/ros/humble/setup.bash
ros2 --help                       # the command exists
python3 -c "import serial; print('serial ok')"
```

And if you did §2.5:

```
python3 -c "import numpy, cv2, ultralytics; from cv_bridge import CvBridge; \
  print('numpy', numpy.__version__, '| opencv', cv2.__version__, '| OK')"
```

If all three print without a traceback, installation is done. Skip ahead to §5 unless you are building the Raspberry Pi setup.

3. Setting up the Raspberry Pi

This is Setup B from §1.3. The repo's helper scripts such as `scripts/driver.sh` expect the Docker container of §2.3, so on the Pi you run the launch files directly instead. Everything else is the same.

3.1 What the Pi does

The Pi does one job: sit next to the arm, hold the USB cable, and run the **driver**, the program that turns joint angles into servo bytes. It does not plan motions and it does not run RViz. A Pi is too slow for comfort at both, and neither needs to be near the robot.

So the split is:

Machine	Runs	Why there
Raspberry Pi	`dofbot_driver`, `robot_state_publisher`	must be physically wired to the arm
Your laptop	MoveIt, RViz, the Python tools	needs a screen and CPU

3.2 Flash the operating system

1. Download **Raspberry Pi Imager** on your normal computer.
2. Insert the microSD card (16 GB or larger).
3. Choose OS → Other general-purpose OS → Ubuntu → **Ubuntu Server 22.04 LTS (64-bit)**.

> It must be **22.04** and it must be **64-bit**. ROS 2 Humble has no packages for other versions, and the 32-bit build will not install. Server, not Desktop. You do not need a GUI on the Pi and it wastes memory the driver could use.

1. Click the gear/settings icon **before writing** and set:
   - hostname, e.g. `dofbot-pi`
   - enable SSH, with password authentication
   - username and password
   - your WiFi network name and password, and your WiFi country
2. Write the card, then put it in the Pi and power on.

3.3 First contact

Give it two or three minutes on first boot, then from your laptop:

```
ssh <your-username>@dofbot-pi.local
```

If `.local` does not resolve, find the Pi's IP address from your router's device list and use that: `ssh youruser@192.168.1.42`.

Then update it:

```
sudo apt update && sudo apt upgrade -y
```

3.4 Install ROS 2 on the Pi

Exactly the steps from §2.2.1, then a **smaller** package set. There is no screen on the Pi, so no RViz:

```
sudo apt install -y \
  ros-humble-ros-base \
  ros-humble-moveit-configs-utils \
  ros-humble-robot-state-publisher \
  ros-humble-xacro \
  ros-humble-control-msgs \
  python3-colcon-common-extensions python3-serial git

echo 'source /opt/ros/humble/setup.bash' >> ~/.bashrc
sudo usermod -aG dialout $USER
```

`ros-humble-ros-base` is ROS without the graphical tools. `moveit-configs-utils` looks like it should not be needed, but the hardware launch file uses it to build the robot description from the URDF (§7.2). Install it or `control.launch.py` will fail on import. If apt cannot find it, `sudo apt install ros-humble-moveit` pulls it in along with a lot you will not use.

Log out and back in so the `dialout` group applies.

3.5 Copy the project to the Pi

From your laptop, in the repo directory:

```
rsync -av --delete \
  --exclude 'build/' --exclude 'install/' --exclude 'log/' \
  --exclude '.git/' --exclude '__pycache__/' \
  ~/ros2_ws/yahboom-robot-arm/ \
  youruser@dofbot-pi.local:~/yahboom-robot-arm/
```

Excluding `build/`, `install/` and `log/` matters: those contain compiled output for **your laptop's** architecture. The Pi is ARM, so it must compile its own. §5.2 explains those folders.

Re-run that same command any time you change code on the laptop. Make it an alias, because you will run it a lot.

3.6 Build on the Pi

```
ssh youruser@dofbot-pi.local
cd ~/yahboom-robot-arm/dofbot_ros2_ws
colcon build --symlink-install
source install/setup.bash
```

The first build takes a few minutes on a Pi 4. If it runs out of memory, build one package at a time with `--executor sequential`.

Add the workspace to the Pi's `.bashrc` so every SSH session has it:

```
echo 'source ~/yahboom-robot-arm/dofbot_ros2_ws/install/setup.bash' >> ~/.bashrc
```

3.7 Plug in the arm and check the port

Connect the arm's USB serial cable to the Pi, and the arm's **DC power supply** to the wall. Then:

```
ls /dev/ttyUSB*
```

You want `/dev/ttyUSB0`. If nothing appears, check `dmesg | tail`. You should see the `ch341` driver claiming the device. No line at all means a cable or power problem, not software.

3.8 Run the driver on the Pi

Press the **K1 button** on the arm's expansion board first. That centers all the servos, which is the pose the driver assumes (§1.5, rule 1). Then:

```
ros2 launch dofbot_bringup control.launch.py port:=/dev/ttyUSB0
```

Look for `Connected to /dev/ttyUSB0` in the output. **No such line means the driver cannot reach the arm.** Every command is silently dropped while ROS reports success. This is the most confusing failure mode in the whole project, so check for that line every time.

Leave this running. Now set up the network so your laptop can reach it.

4. Networking: making the PC and the Pi talk

4.1 The one thing to understand first

ROS 2 has no central server. There is no "master" to point at. Instead, every node **announces itself on the local network**, saying in effect "I publish `/joint_states`, does anyone want it?", and any node that wants it answers directly. This is called **DDS discovery**.

The practical consequence: two machines find each other automatically, but **only if the network lets them broadcast to each other**. Almost every "the Pi can't see my laptop" problem is the network blocking that, not ROS being misconfigured.

Three conditions must hold:

1. Both machines are on the **same network** (same WiFi/subnet).
2. Both have the **same** `ROS_DOMAIN_ID`.
3. Nothing (firewall, VPN, WSL's NAT, guest-network isolation) blocks traffic between them.

4.2 Set the domain ID on both machines

The domain ID keeps your robot from talking to a classmate's robot on the same WiFi. Pick a number between 0 and 101 and use it everywhere.

On the Pi **and** on your laptop:

```
echo 'export ROS_DOMAIN_ID=42' >> ~/.bashrc
echo 'export ROS_LOCALHOST_ONLY=0' >> ~/.bashrc
source ~/.bashrc
```

`ROS_LOCALHOST_ONLY=0` explicitly allows talking off-machine. Some setups default it to 1, which silently confines everything to one computer.

> If you use the Docker container, put these in the container's `.bashrc` too. A variable set in WSL does not reach inside the container.

Verify on each machine:

```
echo $ROS_DOMAIN_ID
```

Both must print the same number. A mismatch produces the exact same symptom as a broken network: total silence, no error.

4.3 Confirm the machines can reach each other at all

Before blaming ROS, test plain networking:

```
# from the laptop
ping dofbot-pi.local

# from the Pi
ping <your-laptop-ip>
```

If ping fails in either direction, stop and fix that first. Common causes: the two devices are on different WiFi bands that the router isolates, a "guest network" that blocks device-to-device traffic, or a VPN on the laptop capturing all traffic.

Then test that ROS's discovery broadcast works. On one machine:

```
ros2 multicast receive
```

On the other:

```
ros2 multicast send
```

The receiving side should print the message. If ping works but multicast does not, your network blocks multicast. See §4.7.

4.4 Special case: WSL2 on the laptop

This one catches everybody. By default WSL2 sits behind a virtual NAT. Your Pi cannot reach into it, so the Pi's topics will never appear in WSL, even though WSL can ping the Pi. It looks like a one-way network, because it is one.

The clean fix is **mirrored networking mode**, which makes WSL share your Windows machine's actual network address. Requires Windows 11 22H2 or newer and WSL 2.0+.

Create or edit `C:\Users\<you>\.wslconfig`:

```
[wsl2]
networkingMode=mirrored
```

Then from PowerShell:

```
wsl --shutdown
```

Reopen WSL. Check with `ip addr`. WSL should now show your real LAN address (e.g. `192.168.1.x`) instead of a `172.x` NAT address.

You may also need to allow inbound traffic to the WSL virtual machine, in an **admin** PowerShell:

```
Set-NetFirewallHyperVVMSetting -Name '{40E0AC32-46A5-438A-A0B2-2B479E8F2E90}' `
  -DefaultInboundAction Allow
```

Because the Docker container is started with `--net=host` (§2.3.2), it shares WSL's network, so fixing WSL fixes the container too.

If you cannot use mirrored mode, do not fight it. Either run the laptop side on native Ubuntu, or go back to Setup A (§1.3) and plug the arm into the laptop.

4.5 Firewall

On the Pi, if you enabled `ufw`:

```
sudo ufw allow from 192.168.1.0/24    # your subnet
```

DDS uses a range of UDP ports that shifts with the domain ID, so allowing the whole local subnet is far more practical than listing ports.

4.6 Verify the two machines are really connected

Start the driver on the Pi (§3.8), then from the laptop:

```
ros2 node list       # expect /dofbot_driver and /robot_state_publisher
ros2 topic list      # expect /joint_states among others
ros2 topic echo /joint_states --once
```

If you see the Pi's nodes and a stream of joint positions, the network is done. Now start MoveIt on the **laptop**:

```
ros2 launch dofbot_bringup moveit.launch.py
```

MoveIt on the laptop sends trajectory goals to the action servers that the driver on the Pi provides (§7.3). Neither side is configured with the other's address, because discovery handles it.

4.7 Network problems and what they mean

Symptom	Cause and fix
`ros2 node list` empty on both machines	`ROS_DOMAIN_ID` mismatch, or `ROS_LOCALHOST_ONLY=1`
Each machine sees only its own nodes	discovery blocked by WSL NAT (§4.4), a VPN, or a firewall
Ping works, multicast does not	some routers/APs block multicast. Switch both to `export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp` (install `ros-humble-rmw-cyclonedds-cpp` on both) and configure a peer list, or use a wired switch
Topics appear, then vanish	WiFi power-saving on the Pi. `sudo iw dev wlan0 set power_save off`
Everything doubles up / weird conflicts	a classmate on your domain ID. Change it
Laggy or stuttering motion	WiFi. The driver is timing-sensitive, so use Ethernet for the Pi if you can

5. ROS workspaces, explained plainly

No robot needed for this section. Read it before §6. It will save you a lot of confusion later.

5.1 What a workspace is

A workspace is **a folder** where you keep robot code and where the build tool puts the compiled result. That is all it is. There is nothing special about it. ROS calls it a workspace. You can think of it as "my project folder".

This project's workspace is:

```
yahboom-robot-arm/dofbot_ros2_ws/
```

It sits *inside* the repo, and is not the repo itself. The repo also has `tools/`, `scripts/`, `config/` and `docs/` which are **not** part of the workspace. They are plain Python scripts and shell scripts you run directly, with no building involved.

5.2 The four folders inside a workspace

After you build once, you will see:

```
dofbot_ros2_ws/
├── src/       <- YOUR CODE. The only folder you edit or commit.
├── build/     <- scratch space the build tool uses. Ignore it.
├── install/   <- the finished, runnable result. ROS reads from here.
└── log/       <- build logs, for when a build fails.
```

The important idea: **you edit `src/`, and the build copies/links the result into `install/`. ROS only ever runs what is in `install/`.**

That explains a beginner mystery: "I changed my file but nothing changed when I ran it." You changed `src/`, but you ran `install/`. You have to build (or use `--symlink-install`, §5.5).

`build/`, `install/` and `log/` are disposable. Delete them any time. A rebuild recreates them. They are in `.gitignore` for exactly that reason, and this is why you exclude them when copying to the Pi (§3.5).

5.3 What "source" means

You will type this constantly:

```
source install/setup.bash
```

All it does is set some environment variables in **this one terminal**, so that the `ros2` command knows where your packages live. It is like adding a folder to your `PATH`.

Three things follow from that:

- It applies to **one terminal only**. Open a new terminal, do it again.

- It does **not** survive a reboot, which is why people put it in `~/.bashrc` to run automatically.
- You need **two** sources: `/opt/ros/humble/setup.bash` for ROS itself, then `install/setup.bash` for this project. The second layers on top of the first.

The most common beginner error in ROS is `ros2: command not found` or `Package 'dofbot_bringup' not found`. Nine times out of ten it is a terminal you forgot to source.

In this project's container, `scripts/one_time/setup_container.sh` (§2.3.5) adds both lines to `.bashrc`, and every `scripts/*.sh` sources them anyway, so you rarely have to think about it.

5.4 What a package is

Inside `src/`, code is organised into **packages**. A package is a folder with one job and a `package.xml` file that names it and lists what it depends on. This project has five:

```
dofbot_ros2_ws/src/
├── dofbot_description/    the robot's shape: URDF, 3D meshes
├── dofbot_driver/         talks to the servos over USB serial
├── dofbot_moveit_config/  settings for the motion planner
├── dofbot_bringup/        launch files, the things you actually run
└── dofbot_vision/         camera, YOLO detection, picking
```

Why split it up? So each part can be replaced without touching the others. When the physical camera path stopped working, only `dofbot_vision`'s camera node was swapped and nothing downstream noticed or cared.

5.5 Building with colcon

`colcon` is the build tool. From the workspace directory:

```
cd ~/ros2_ws/yahboom-robot-arm/dofbot_ros2_ws
colcon build --symlink-install
source install/setup.bash
```

What `--symlink-install` does, and why you want it. Normally the build *copies* files into `install/`. With this flag it makes shortcuts (symlinks) instead. So when you edit a Python file or a launch file, the change takes effect immediately, with no rebuild needed. You still must rebuild after adding a new file, changing `package.xml`, or touching C++.

Other flags you will use:

```
colcon build --symlink-install --packages-select dofbot_vision   # just one package
colcon build --symlink-install --executor sequential             # low-memory machines (the Pi)
```

In the container, §2.3.5 defined a shortcut that always builds in the right place:

```
ros-build                          # equals: cd dofbot_ros2_ws && colcon build --symlink-install
ros-build --packages-select dofbot_vision
```

5.6 Rules for this repo

1. **Always build in `dofbot_ros2_ws/`.** If you run `colcon build` in the repo root, you create a *second* `build/` + `install/` tree there, and from then on you will have two versions of everything and no way to tell which one is running. If you have already done it: `rm -rf build install log` at the repo root.
2. **When a build acts strange, clean it.** Stale artifacts cause errors that make no sense:

   ```
   cd dofbot_ros2_ws && rm -rf build install log && colcon build --symlink-install
   ```

3. **After pulling changes from git, rebuild.**
4. **Every new terminal must be sourced** (§5.3).

6. Building the project and running the simulation

This is the checkpoint. **Do not connect the real arm until this section works.** Simulation catches install problems safely. Hardware catches them by hitting your desk.

6.1 Build

```
# Path A (native Ubuntu):
source /opt/ros/humble/setup.bash
cd ~/ros2_ws/yahboom-robot-arm/dofbot_ros2_ws

# Path B (container):  docker exec -it dofbot bash
#                      cd /root/yahboom-robot-arm/dofbot_ros2_ws

rm -rf build install log          # start clean the first time
colcon build --symlink-install
source install/setup.bash
```

Expect `Summary: 5 packages finished`. Warnings about `setuptools` deprecation are normal and harmless.

6.2 Run the simulation

```
scripts/sim.sh                    # in the container
# or, anywhere:
ros2 launch dofbot_bringup demo.launch.py
```

An RViz window opens with a 3D model of the arm.

> **Never run `sim.sh` at the same time as `driver.sh` or `moveit.sh`.** They create nodes and action servers with the same names, and the result is two programs fighting over one robot. Ctrl-C one before starting the other.

6.3 What that one command started

Understanding this makes RViz much less mysterious. Six programs launched:

Program	Job
`robot_state_publisher`	reads the URDF, works out where every link is given the joint angles, and broadcasts those positions so RViz can draw them
`move_group`	MoveIt's brain: given a goal, finds a collision-free path
`ros2_control_node`	the **fake** hardware. It pretends to be servos and reports back exactly whatever it was told, which is what makes simulation work with no robot
`arm_controller`	takes a planned path and feeds it out over time
`gripper_controller`	the same, for the claw

Program	Job
`rviz2`	the 3D window

In simulation, ros2_control drives that fake hardware. On **real** hardware it does not run at all, and the driver replaces the whole bottom half (§7.9). That difference is the most important thing to understand about this project.

6.4 Your first planned motion

In the RViz **MotionPlanning** panel, **Planning** tab:

1. Set **Planning Group** to `arm`. (The other group, `gripper`, is just the claw joint.)
2. Give it a goal, either:
   - drag the orange ball at the end of the arm to a new spot, or
   - set **Goal State** to `<random valid>`
3. Click **Plan**. An animated preview of the path plays.
4. Click **Execute**. The virtual arm moves.

If the virtual arm moves, your environment is correct. That is the checkpoint passed.

6.4.1 Why dragging the ring does nothing useful

The orange marker has position arrows and rotation rings. Only the arrows work. The arm has 5 joints, and positioning a gripper in space *with a chosen orientation* needs 6. So MoveIt here is configured `position_only_ik: true`, meaning "get the gripper to that point, and take whatever angle you get".

This is not a bug or a limitation to fix. It is what a 5-joint arm is. It matters later: §12 works entirely around choosing the gripper's approach angle deliberately, because MoveIt will not do it for you.

6.5 Speed

Default speed scaling is 0.5 of the joint limits. Two dropdowns in the Planning tab, **Velocity Scaling** and **Accel Scaling**, change it per plan. Set them to 0.1 before your first real-hardware motion.

The defaults live in `dofbot_ros2_ws/src/dofbot_moveit_config/config/moveit_params.yaml`.

7. How the system works, end to end

This section answers one question: when you click Execute, what happens all the way down to the wire? You do not need to memorize it, but knowing the chain is what lets you debug anything.

7.1 The chain

```
You, in RViz / a Python tool
        │
        │  "put the gripper at x=0.16, y=0.02, z=0.09"
        ▼
MoveIt (or the tools' own IK, §7.6)
        │  a TRAJECTORY: a list of joint angles with timestamps
        │  sent as a FollowJointTrajectory action goal
        ▼
dofbot_driver                          <-- the translator
        │  radians -> degrees -> tick counts -> bytes
        ▼
USB serial, 115200 baud, /dev/ttyUSB0
        │
        ▼
Expansion board firmware
        │  smoothly interpolates each servo to its target
        ▼
Six servos
```

7.2 Step 1: describing the robot (URDF)

Everything starts from `dofbot_ros2_ws/src/dofbot_description/urdf/dofbot.urdf.xacro`. It is an XML file listing every **link** (a rigid part) and every **joint** (a connection that moves), with exact measurements:

```
<joint name="arm_joint2" type="revolute">
    <origin xyz="0 0 0.0405" rpy="-1.5708 0 0" />
    <parent link="arm_link1" />
    <child  link="arm_link2" />
    <axis xyz="0 0 -1" />
    <limit lower="-1.5708" upper="1.5708" effort="100" velocity="1" />
</joint>
```

Read that as: joint 2 connects link 1 to link 2, sits 40.5 mm above it, rotates about one axis, and can turn ±1.5708 radians (±90°).

This one file is the source of truth for RViz's drawing, MoveIt's planning, and the arm's real dimensions. The numbers in the tools' own kinematics (§7.6) were taken straight out of it.

Angles are in radians everywhere in ROS. 90° = 1.5708 rad. Get used to it. The driver is the only place degrees appear.

7.3 Step 2: the command, what a trajectory is

Whether it comes from MoveIt or from `tools/arm_client.py`, the arm is commanded with a **FollowJointTrajectory action goal**. Stripped down, it is a list of waypoints:

```
joint_names: [arm_joint1, arm_joint2, arm_joint3, arm_joint4, arm_joint5]
points:
  - positions: [0.05, -0.30, 0.44, -0.90, 0.0]   time_from_start: 0.1 s
  - positions: [0.10, -0.35, 0.50, -0.95, 0.0]   time_from_start: 0.2 s
  - ...
  - positions: [0.52, -0.60, 0.80, -1.20, 0.0]   time_from_start: 2.0 s
```

"Be at these angles at these times." An **action** (rather than a plain message) because it is a long-running request that reports back when it finishes, and can be cancelled.

The driver provides two of these action servers, and this is what MoveIt connects to:

- `/arm_controller/follow_joint_trajectory` for the five arm joints
- `/gripper_controller/follow_joint_trajectory` for the claw

7.4 Step 3: the driver throws most of the waypoints away

This surprises people, so it is worth explaining.

The servo firmware already knows how to move smoothly: you tell it "go to position P over 2000 ms" and it does its own acceleration ramp. If you instead send it 20 tiny commands, **each new command restarts that ramp**, and the arm stutters and lags.

So `driver_node.py` downsamples. The `min_segment_ms` parameter (default **3000 ms**) means: keep waypoints at least 3 seconds apart, plus always the final one. Since a typical motion is 2 to 3 seconds, in practice **only the last point of each motion survives**. That is one serial command per motion, letting the firmware interpolate the whole thing. That is exactly how Yahboom's own demo code drives this board.

You will see it in the driver's log:

```
Trajectory goal: 5 joints (arm_joint1, ...), 30 points over 2.00s -> 1 segments
```

30 points in, 1 segment out.

7.5 Step 4: radians → servo degrees

`_rad_to_driver_deg()` in `driver_node.py`.

Arm joints. The URDF's zero is the arm straight up, while the servo's centre is 90°:

```
servo_degrees = degrees(radians) + 90
```

So `arm_joint1 = 0.5236 rad` (30°) becomes servo angle **120°**.

The gripper is different. Its command range is 0 rad (open) to −1.54 rad (closed), mapped onto servo angles 0° (open) to 180° (closed):

```
t = (position + 1.54) / 1.54           # 0..1
servo_degrees = 180 + t * (0 - 180)
```

So `-1.1 rad` → t = 0.2857 → **128.57°**.

> **Gripper convention, worth writing on your hand:** `0.0` = wide open, `-1.54` = fully closed. More negative = tighter. The chess tools use `-1.1` for open and `-1.42` for a grip. (RViz renders the claw mirrored relative to this, so the driver deliberately publishes a flipped value for display only. The commands themselves are as described here.)

7.6 Where the joint angles come from: two different IK paths

Inverse kinematics (IK) = "I want the gripper *there*, so what angle does each joint need?"

This project has two answers:

Path	Used by	How
MoveIt's numeric IK	RViz drag-and-plan (§6.4)	KDL solver, iterative, randomized
Closed-form IK	all the Python tools in `tools/`	direct trigonometry in `tools/arm_client.py`

The tools stopped using MoveIt's solver for two concrete reasons:

1. It positioned the **wrist frame**, not the fingertips. As the gripper tilt changed with height, the actual touch point slid sideways. That is fatal when you are trying to grab a chess piece 26 mm wide.
2. It returned a **different solution every call** for the same target, because it starts from a random seed.

`arm_client.py` solves the geometry directly for the **grasp point between the fingertips**, scanning gripper tilt from vertical outward and returning the most upright grip that works. Same input, same output, every time. The arm is treated as planar: joint 1 swings the whole plane around, joints 2 to 4 work within it, joint 5 stays at zero.

7.7 Step 5: servo degrees → tick counts

Each servo wants a raw number, not degrees. `_servo_pos()`:

Servo	Formula	Note
1, 6	`(3100-900) * angle/180 + 900`	900 to 3100 over 180°
2, 3, 4	`(3100-900) * (180-angle)/180 + 900`	**inverted**, mounted facing the other way
5	`(3700-380) * angle/270 + 380`	270° range, different limits

Example: servo 1 at 120° → `2200 × 120/180 + 900` = **2366**.

7.8 Step 6: the actual bytes

The packet format is:

```
FF FC [len] [cmd] [payload ...] [checksum]
```

- `FF FC` is the fixed header. `FF` starts the packet, `FC` says it is for the arm board.

- `len` counts every byte from `len` itself through the checksum.
- `cmd` of `0x1D` writes all six servos at once. `0x10 + id` writes one.
- `payload` for `0x1D` is six 16-bit positions, high byte first, then a 16-bit duration in milliseconds.
- `checksum` is the sum of the bytes from `len` onward, kept to 8 bits with `& 0xFF`. The board rejects a packet whose sum does not match.

7.8.1 A complete worked example

Command: **joint 1 to 30°, everything else centered, gripper at −1.1 rad, over 1000 ms.**

Servo	Command	Driver degrees	Tick count	Hex
1	0.5236 rad	120.00	2366	`09 3E`
2	0 rad	90.00	2000	`07 D0`
3	0 rad	90.00	2000	`07 D0`
4	0 rad	90.00	2000	`07 D0`
5	0 rad	90.00	1486	`05 CE`
6	−1.1 rad	128.57	2471	`09 A7`

Duration 1000 ms = `03 E8`. Length = 1 (len) + 1 (cmd) + 12 (positions) + 2 (time) + 1 (checksum) = 17 = `0x11`.

The 19 bytes that go down the wire:

```
FF FC 11 1D 09 3E 07 D0 07 D0 07 D0 05 CE 09 A7 03 E8 68
└──┬──┘ │  │  └──────────── six positions ─────────────┘ └┬┘ └┬┘
 header │ cmd                                            time  checksum
        len
```

You can watch this happen for real: `tools/serial_wiggle.py` sends exactly this by hand, with no ROS involved at all. If the arm moves when you run it, your hardware is fine and any problem is above the driver.

7.9 Why hardware and simulation take different routes

```
SIMULATION                          REAL HARDWARE
──────────                          ─────────────
MoveIt                              MoveIt (or a Python tool)
   │                                   │
   ▼                                   │
ros2_control                           │
   │  (100 Hz position stream)         │  (FollowJointTrajectory action)
   ▼                                   ▼
mock hardware                       dofbot_driver
   │                                   │
   ▼                                   ▼
RViz shows the result               real servos
```

On hardware, `ros2_control` is **not used at all**. The driver serves the trajectory actions itself. `ros2_control` streams joint positions at 100 Hz, which over a serial link means a new command every 10 ms, each one restarting the servo's own ramp (§7.4). The arm pulses and lags. Handing the whole trajectory to the driver and letting the servos interpolate avoids that entirely.

7.10 The one thing to remember: it is open loop

The driver cannot read the servos. No known command on this board reports a servo's real position. (`tools/probe_protocol.py` exists solely to hunt for one, by sweeping unknown command bytes to see if anything answers.)

So when the driver publishes `/joint_states`, it is not reporting where the arm *is*. It is echoing back **what it last told the arm to do**.

Consequences you must plan around:

- If the arm is blocked, or a servo stalls under load, ROS will never know. It will keep reporting success.
- If the arm did not start centered, every command is offset by that error for the entire session. Hence §1.5 rule 1, and the K1 button.
- On its very first motion the driver performs a slow **startup sync** (`startup_time_ms`, default 4000 ms), gliding all servos to the first commanded pose so a mismatch never becomes a snap. This is why every tool waits 4 seconds after connecting before it starts moving.

7.11 The shortcut command path

The driver also subscribes to a `/target_joints` topic and moves to whatever `JointState` you publish there. It is there for quick one-off scripting, with no action client and no trajectory to build. MoveIt does not use it, and it is ignored while a trajectory action is running. Prefer the actions for anything real.

8. Bringing up the real arm

Do §6 first.

8.1 Physical checklist, every session

1. **DC power supply connected and switched on.** USB does not power the servos.
2. **Arm posed roughly straight up.** Press **K1** on the expansion board to center the servos.
3. **Nothing within the arm's reach** that you mind being hit.
4. USB serial cable connected.

8.2 WSL2 only: hand the USB device to Linux

Windows owns USB devices by default. `usbipd` lends one to WSL.

In PowerShell (**admin** needed for `bind`, once per device ever):

```
winget install usbipd                # one time
usbipd list                          # find "USB Serial", VID:PID 1a86:7523
usbipd bind --busid <BUSID>          # one time, admin
usbipd attach --wsl --busid <BUSID>  # EVERY reboot or replug
```

Or let the repo do it, from WSL:

```
scripts/container.sh      # start the Docker container
scripts/usb.sh            # find, attach, and keep watching the device
```

`scripts/usb.sh` also starts an **auto-attach watcher**, which matters: the CH340 adapter drops out on cable wiggles and power brownouts, and re-appears, sometimes as `/dev/ttyUSB1` instead of `ttyUSB0`. The watcher re-attaches it, and the driver picks up whatever `ttyUSB*` is present.

Verify:

```
ls /dev/ttyUSB*
```

8.3 Start the driver in terminal 1

```
docker exec -it dofbot bash
scripts/driver.sh
```

The script refuses to start if there is no `/dev/ttyUSB*`, because a driver that cannot reach the arm is worse than no driver at all (§3.8). It then asks you to confirm the K1 pose, and launches.

Watch for `Connected to /dev/ttyUSB0`**.** No line, no arm.

Leave this terminal open, because it is your log. If the arm misbehaves, this is the first place to look.

Without the scripts, the equivalent is:

```
ros2 launch dofbot_bringup control.launch.py port:=/dev/ttyUSB0
```

Useful parameters:

Parameter	Default	Effect
`port`	`/dev/ttyUSB0`	serial device
`startup_time_ms`	4000	how slowly the first sync move happens
`max_speed_deg_s`	120	hardware speed cap, applied to everything
`min_segment_ms`	3000	waypoint downsampling (§7.4)

8.4 Start MoveIt in terminal 2

```
docker exec -it dofbot bash
scripts/moveit.sh             # add --no-rviz for headless
```

8.5 Check the stack in terminal 3

```
scripts/status.sh
```

It reports the serial device, which processes are alive, and whether the key topics and action servers exist. Other views: `scripts/status.sh nodes`, `topics`, `actions`, `controllers`, or `hz /joint_states`.

8.6 First real motion

In RViz: set **Velocity Scaling to 0.1**, plan a *small* move, and Execute.

Watch for: smooth motion, no USB errors in terminal 1, arm ends up roughly where the preview said. If all three hold, you can move on to §9.

If the arm jumps violently, it did not start centered. Power cycle, press K1, and restart the driver.

9. Teleop: driving the arm with the keyboard

Teleop ("teleoperation") is direct human control: you press a key, the arm moves a little. It is the fastest way to build intuition for what the arm can and cannot physically do, and it is how you find the numbers you will later put in config files.

The tool is `tools/teleop_key.py`. It works against **either** the real driver (§8) or the simulation (§6.2), because both provide the same trajectory actions.

9.1 Start it

With either the driver or the sim already running, in another terminal:

```
cd /root/yahboom-robot-arm      # or ~/ros2_ws/yahboom-robot-arm
python3 tools/teleop_key.py
```

It waits 4 seconds for the driver's startup sync (§7.10), prints the help and the current pose, and then listens for single keypresses. You do **not** press Enter after each key.

9.2 The keys

```
JOINT mode                      CARTESIAN mode
  1..5  select a joint            w / s   +x / -x   (forward / back)
  j     selected joint -= step    a / d   +y / -y   (left / right)
  k     selected joint += step    q / e   +z / -z   (up / down)

Both modes
  m     switch between modes      o     open the gripper
  h     go home (straight up)     c     close the gripper
  p     print the current pose    [ / ] gripper by a small step
  + / - make the step bigger/smaller
  ?     show this help
  x  or Ctrl-C   quit
```

After every keypress it prints a status line:

```
  JOINT [joint2] step 5deg | joints   +0.0  -25.0  +40.0  -70.0   +0.0 | grasp x=+0.171 y=+0.000 z=+0.088
tilt +125deg | grip -1.10
```

That line is worth reading: it gives you the joint angles **and** where the fingertips are in robot coordinates. The two modes use it differently.

9.3 Joint mode, start here

Each key moves **one motor**. There is no cleverness in between: press `k` and joint 2 rotates 5°. Nothing can be "unreachable", so nothing can fail.

Use joint mode to:

- learn which joint is which (press `1`, then `j` / `k`, and watch)

- feel out the ±90° limits
- get the arm out of an awkward pose

9.4 Cartesian mode, thinking in space rather than joints

Press `m`. Now the keys move the **grasp point**, the spot between the open fingertips, through space:

- `w` and `s` move it away from and toward the robot's base (**x**)
- `a` and `d` move it to the robot's left and right (**y**)
- `q` and `e` move it up and down (**z**)

Behind each keypress, the IK from §7.6 works out all five joint angles. This is the same code the chess and picking tools use, so if a spot is awkward here, it will be awkward for them too.

When a target cannot be reached you get:

```
!! (+0.280, +0.000, +0.160) is out of reach - stepping back
```

and nothing moves. Nothing is broken. You asked for a point outside the arm's envelope. Come back with `s` and try lower.

Coordinate frame. The origin is the base's rotation axis, at table level. +x points away from the robot (its "forward"), +y to its left, +z up. Every number in `config/board.yaml` and in the vision calibration uses this same frame, so what you learn here carries over.

9.5 What to use it for

Teleop is how you measure the numbers other parts of the project need:

To find	Do this
the height where fingertips touch the table	cartesian mode, `e` down in 1 mm steps (`-` to shrink the step). The z where they touch is your table height
a good grasp height for an object	put the object under the gripper, descend until the fingers straddle it. That z is `grasp_z`
a grip that holds without straining	`[` in small steps until the object is held firmly. That value is `grip_closed`
whether a spot is reachable at all	drive there and see

9.6 Safety notes specific to teleop

- **Each keypress finishes before the next is read.** Anything typed while the arm is moving is thrown away deliberately, because otherwise holding a key down would queue twenty moves and the arm would keep going long after you let go.
- **Start with small steps.** The default is 5° / 10 mm. Press `-` a couple of times when working near the table.
- **The gripper can crush things,** including itself against the table. Move down in small steps.

- Quitting leaves the arm holding its last pose. It does not go limp and it does not go home. Press `h` first if you want it upright.

10. Checking trajectories in RViz before running for real

The arm has no obstacle model, no force sensing, and no way to report that it hit something (§7.10). The preview in RViz is your only chance to catch a bad path before it happens.

10.1 Plan, look, then Execute

The **Plan** and **Execute** buttons are separate for a reason. Use **Plan** alone. RViz animates the proposed path without sending anything to the robot. Only if the animation looks right do you press **Execute**.

`Plan & Execute` is one button that does both. Avoid it on hardware until you trust a particular motion.

10.2 Make the preview easier to read

In the RViz **Displays** panel on the left, open **MotionPlanning → Planned Path**:

Setting	Set it to	Why
Show Trail	ticked	draws every waypoint at once, so you see the whole swept path instead of a moving robot
Loop Animation	ticked	replays continuously so you can study it
State Display Time	`0.05 s` or slower	slows the animation down
Trajectory Topic	`/display_planned_path`	should already be set

With **Show Trail** on, a path that dives through your table is obvious at a glance. Without it, you have to catch the moment.

10.3 What to look for

1. **Does it go through the table?** The table is not in the planning scene. Any path passing below the arm's base plane will hit it.
2. **Does it swing wide?** With no obstacles to avoid, the planner takes whatever path is mathematically easy, which is sometimes a big arc through where your hand is.
3. **Does the elbow flip?** Watch for the arm suddenly reconfiguring mid-path. That is jerky and hard on the servos. Re-plan, and you often get a different, cleaner path, since the planner is randomized.
4. **Does the gripper end at a usable angle?** Because IK is position-only (§6.4.1), the final gripper angle is whatever the solver happened to pick. It may end up horizontal, which cannot grab anything off a table.

10.4 Re-plan freely

Press **Plan** again on the same goal. OMPL (the default planner) is randomized, so you get a different path each time. If the first one looks bad, ask again. This is normal practice, not a workaround.

For predictable straight lines, switch the **Planning Pipeline** dropdown to **Pilz**, which does simple point-to-point and linear moves instead of searching.

10.5 Check reachability without any robot at all

Before planning anything, you can ask "can the arm even get there?" offline, with no driver, no simulation and no RViz:

```
python3 tools/reach_check.py --hover-z 0.10 --grasp-z 0.053
```

It runs the closed-form IK (§7.6) over all 64 chess-board squares and prints a map:

```
  #  = reachable at both travel and grasp height
  o  = travel height only        ^ = grasp height only
  .  = cannot reach
```

Use it to decide where to put things *before* you build the setup. It takes a second to run.

10.6 Rehearse whole sequences in simulation

The Python tools do not care whether they are driving real servos or the mock ones. So you can rehearse a complete run with no hardware:

```
# terminal 1
scripts/sim.sh
# terminal 2
cd /root/yahboom-robot-arm
python3 tools/chess_game.py --self-play --hover-z 0.06 --grasp-z 0.05
```

The engine plays itself and the virtual arm executes every move. Any crash, any unreachable square, any logic error surfaces here where the consequence is an error message instead of a bent servo horn.

Similarly, `python3 tools/teleop_key.py` against the sim lets you practise the keys (§9) before touching the real arm.

10.7 Pre-flight checklist

Before the first hardware run of any new motion:

- ☐ It plans successfully in simulation
- ☐ The trail shows no path through the table or through you
- ☐ `reach_check.py` says the targets are reachable
- ☐ Velocity Scaling is 0.1 to 0.2
- ☐ The workspace is clear
- ☐ Your hand is near the power switch

11. Vision: OpenCV pick and place

The goal: a camera sees an object, and the arm goes and picks it up.

Three problems have to be solved, in this order:

1. **Get camera frames into ROS** (§11.2)
2. **Find the object in the image.** YOLO gives you a pixel location (§11.3)
3. **Turn pixels into robot coordinates.** This is the calibration step, and it is the one that decides whether the arm grabs the object or the air next to it (§11.4)

11.1 The core idea: a homography

A camera gives you a **pixel**: "the cup is at x=320, y=240 in the image." The arm needs **meters**: "the cup is 18 cm forward and 3 cm left of my base."

Normally converting between the two is hard, because a pixel corresponds to a whole *ray* going out from the camera, so you cannot tell how far away something is from one image.

One assumption lets us avoid that: **everything sits flat on one table.** With that, every pixel maps to exactly one point on the table surface, and the whole conversion collapses into a single 3×3 matrix called a **homography**:

```
[ x ]       [ u ]          u, v = pixel coordinates
[ y ] = H · [ v ]          x, y = table coordinates in meters
[ w ]       [ 1 ]          divide x and y by w at the end
```

Find those 9 numbers once and every future pixel converts instantly.

What breaks a homography:

- Moving the camera, even slightly. Recalibrate.
- Moving the table, or changing its height.
- Objects of noticeably different heights. The taller the object, the more its top appears shifted. This is why picking targets should be short and roughly uniform.

11.2 Getting camera frames into ROS

11.2.1 Native Linux

Plug the camera in and run:

```
ros2 run v4l2_camera v4l2_camera_node \
  --ros-args -p video_device:=/dev/video0 -p image_size:=[640,480]
```

11.2.2 WSL2, where the camera cannot be passed through

Do not spend an evening on this: **usbipd cannot forward a USB webcam**. Webcams stream using isochronous USB transfers, which USB/IP does not implement. The camera will *appear* in WSL and then never deliver a single frame, while `dmesg` fills with `vhci ... Not yet implemented`.

The workaround is to leave the camera owned by Windows and send the video over the network instead:

On Windows:

```
winget install ffmpeg
ffmpeg -list_devices true -f dshow -i dummy     # find your camera's name

# from the repo (reachable at \\wsl$\...\yahboom-robot-arm):
.\scripts\windows\stream_camera.ps1
.\scripts\windows\stream_camera.ps1 -Camera "HD Webcam"   # if not "USB Camera"
```

Click **Allow** on the firewall prompt. Leave it running, because the script restarts ffmpeg after each client disconnect, which it needs to do because ffmpeg exits when the reader goes away.

In the container:

```
scripts/camera.sh
# equivalently:
ros2 run dofbot_vision stream_camera \
  --ros-args -p url:=http://host.docker.internal:8090/cam.mjpg
```

11.2.3 Verify, either way

```
ros2 topic hz /image_raw          # want roughly 30
ros2 run rqt_image_view rqt_image_view
```

If the image is blurry, twist the lens barrel. These cameras are manual-focus and ship badly adjusted. Set focus at your working distance, about 20 to 40 cm.

11.3 Object detection with YOLO

YOLO is a neural network that finds objects and draws boxes around them. The bundled `yolov8n.pt` is the smallest pretrained model. It knows 80 everyday classes (cup, bottle, cell phone, ...) and runs at a few frames per second on CPU, which is plenty here.

```
cd ~/ros2_ws/yahboom-robot-arm      # so yolov8n.pt is found
ros2 launch dofbot_vision yolo.launch.py \
  image_topic:=/image_raw model:=yolov8n.pt device:=cpu
```

Two outputs:

- `/detections` carries JSON with the class name, confidence, and `bbox_xyxy`, the box corners in pixels
- `/detections/image` is the image with boxes drawn, for `rqt_image_view`

Check it works by holding up a cup:

```
ros2 topic echo /detections
```

The object's pixel centre is the middle of the box: `u = (x1+x2)/2`, `v = (y1+y2)/2`. That is the number the homography consumes.

11.4 Calibrating the homography

Two ways. Use the first.

11.4.1 The good way: the chessboard

`tools/calibrate_camera.py` finds all 49 inner corners of the printed chess board automatically, pairs each with its known position from `config/board.yaml`, and fits the homography in one least-squares step. 49 correspondences instead of 4, no manual measuring, far more accurate.

Requirements: the camera fixed rigidly and seeing the whole board, and the board **empty** and evenly lit.

```
scripts/camera.sh          # terminal 1, frames flowing
scripts/homography.sh      # terminal 2
```

Options: `--rotate 90` if your camera is mounted sideways, or `--image /path/frame.png` to work from a saved photo. It writes an annotated check image to `runs/calib_check.png`. **Open it** and confirm the detected corners really sit on the corners.

11.4.2 The manual way: four points

For scenes with no chess board:

1. Fix the camera. Any later movement invalidates everything.
2. Put a small object at **4 well-spread, non-collinear spots**. Think of the corners of a large rectangle, not a line and not a tiny cluster.
3. For each spot, note the pixel centre from `/detections`, and measure the real position from the arm's base axis with a ruler (x forward, y left, in meters).
4. Compute:

```
python3 tools/compute_homography.py \
  --image "u1,v1;u2,v2;u3,v3;u4,v4" \
  --base  "x1,y1;x2,y2;x3,y3;x4,y4"
```

11.4.3 Install the result

Either way, paste the printed matrix into the `homography:` block of `dofbot_ros2_ws/src/dofbot_vision/config/picking.yaml`, then:

```
colcon build --symlink-install --packages-select dofbot_vision
```

11.5 Running the picker

```
ros2 launch dofbot_vision pick.launch.py
```

The `pick_from_detections` node ties it together: read a detection → convert the pixel to base coordinates → ask for IK → send the trajectory → approach, close the gripper, lift, optionally place.

> **Out of the box, `picking.yaml` contains a placeholder homography.** The arm will run a complete, confident pick sequence at coordinates that are roughly wrong. Treat that first run as an integration test that proves the plumbing works, and keep your hand near the power switch.

11.5.1 The safe tuning order

`grasp_z` starts at `0.12`, deliberately high.

1. Run with `grasp_z: 0.12`. The gripper should stop **directly above** the object without touching it. If it is not above the object, your calibration is wrong. Fix that before lowering anything.
2. Only once it is centered above, lower `grasp_z` in steps: 0.10 → 0.08 → 0.06, checking each time.
3. Tune `gripper_closed` for grip strength.

Nothing in the software knows where your table is. If you set `grasp_z` below the table surface, the arm will drive the gripper into it and keep pushing, because it cannot tell (§7.10).

11.5.2 Useful settings in `picking.yaml`

Setting	What it does
`target_classes`	only pick these (e.g. `"cup,bottle"`). Empty means anything
`min_confidence`	ignore weak detections (default 0.2)
`pick_once`	stop after one pick. Leave this `true` while tuning
`cooldown`	seconds between picks
`place_x` / `place_y`	set both to make it deposit objects at a fixed spot, which gives you a pick-and-place demo
`approach_z` / `grasp_z` / `lift_z`	heights for each phase
`gripper_open` / `gripper_closed`	claw positions (§7.5)

11.6 Training YOLO on your own objects

The stock model knows cups and bottles, not chess pieces or your lab's parts. Two helper scripts support a custom model:

```
python3 tools/labelme_to_yolo.py --base <folder> --classes classes.txt
python3 tools/prepare_yolo_dataset.py --base <folder> --out <dataset>
```

Label images with LabelMe, convert them, split into train/val, then train with `ultralytics` and point `model:=` at your resulting `.pt` file. The rest of the pipeline is unchanged.

12. The chess demo

This is the project's most developed capability, and the best worked example of everything above. `docs/demo_runbook.md` is the operational checklist. This is the orientation.

12.1 What it does

The robot plays White on a printed 26 mm board. Stockfish chooses its moves, and the arm physically picks up and places the pieces, including removing captured pieces to a discard pile and moving the rook when it castles. You type your own moves.

The arm can only physically reach about **ranks 1 to 4**, so at startup the program maps every reachable square and restricts the engine to moves it can actually execute. When nothing legal is reachable, it announces its move and asks you to make it by hand.

12.2 Print the board

```
python3 tools/gen_board.py --paper a3 --marker-mm 20 --border-mm 10 \
  --out /root/yahboom-robot-arm/board_a3
```

Print `board_a3.pdf` at **100% / Actual size**, not "fit to page", then **check with a ruler that one square is 26 mm.** Everything downstream assumes it. Trim the paper outside the grey border (otherwise the near margin fouls the robot's base) and glue it to something flat.

12.3 Place the board

Rank 1 nearest the robot, centered on the arm's forward direction, with the first grid line about 12 cm from the base's **rotation axis** (not the housing edge):

```
    a8 ............... h8      far side (yours)
    a1 ............... h1      rank 1, ARM SIDE
             ~12 cm
          [robot base]
```

Trace round the board with a pencil or tape. It will get nudged during play, and every nudge invalidates the calibration. The outline is what lets you put it back without recalibrating.

12.4 Calibrate

`config/board.yaml` is the single source of truth for where the board is. Every tool reads it, and command-line flags override it.

```
a1: [0.135, 0.080]   # centre of square a1 in meters (x forward, y left)
square: 0.026        # square size, ruler-check the print
yaw_deg: -90.0       # which way the files a->h run
mirror: false        # flip if hovers land mirrored
```

The calibration loop:

1. `reach_check.py` runs offline and confirms the placement is sensible before you touch anything.
2. `hover_test.py` hovers the arm over named squares so you can see the error:

   ```
   python3 tools/hover_test.py --gripper -1.0 a1 h1 d4 e3
   ```

   Consistently off in one direction → shift `a1`. Files and ranks swapped or mirrored → fix `yaw_deg` / `mirror`.
3. **§9 teleop** is where you find `grasp_z`, the fingertip gripping height, and `grip_closed`, the grip that holds firmly without straining.
4. `place_test.py` picks a piece and puts it back on the *same* square. Wherever it lands relative to where you put it is exactly the systematic error. Measure it with a ruler and record it as a per-square offset anchor in `board.yaml`.

12.5 Play

```
python3 tools/chess_game.py --skill 3
```

- Moves are typed as SAN (`e5`, `Nf6`) or UCI (`e7e5`); `quit` resigns
- `--skill 0..20` sets engine strength
- `--move-time 3` slows the arm down for demonstrations
- `--fen "<position>"` resumes a game
- `--no-arm` runs the game logic with no robot at all

Reachable squares are cached in `runs/reach_cache.json` and recomputed automatically whenever the geometry or heights change.

12.6 Why this demo is harder than it looks

Worth reading even if you never play chess with it. Each of these is a robotics problem the tools already handle, and knowing they exist tells you what you are looking at when something goes subtly wrong:

- **Grasping needs a tilted approach.** Coming straight down makes the claw poke the top of a piece. It has to come in at an angle and wrap around.
- **The tilt stays locked during the descent.** The required angle changes with height at long reach, so a descent that re-solves the tilt each step sweeps the fingers in an arc around the piece it was about to grab.
- **Picking and placing at different tilts shifts the piece.** A piece held at one angle and released at another lands offset by a predictable amount, which the code calculates and cancels.
- **The claw drags pieces to its own centre when it grips**, so where a piece lands is not where you commanded, hence `place_test.py`.
- **The approach angle comes from the *nominal* square centre**, not the corrected one. Otherwise adjusting a square's calibration offset would move the approach angle, which moves the fingertip, which changes the offset you needed, and the calibration never converges.

13. Reference: every script and tool

13.1 Host scripts (run in WSL, outside the container)

Script	Purpose
`scripts/container.sh`	start / stop / status the Docker container
`scripts/usb.sh`	attach the arm's USB serial to WSL, with an auto-reattach watcher
`scripts/windows/stream_camera.ps1`	(Windows) MJPEG camera streamer for the WSL bridge

13.2 Container scripts (run inside the container)

Script	Purpose
`scripts/driver.sh`	the arm driver, runs in the foreground so its log stays visible
`scripts/moveit.sh`	MoveIt + RViz for real hardware (`--no-rviz` for headless)
`scripts/sim.sh`	full simulation, no hardware (`--no-rviz` too)
`scripts/camera.sh`	camera bridge → `/image_raw`
`scripts/homography.sh`	camera→base calibration from the chess board
`scripts/status.sh`	health check for the whole stack
`scripts/one_time/setup_container.sh`	provision the container, set up `.bashrc` and `ros-build`
`scripts/one_time/run_moveit_setup.sh`	re-run the MoveIt Setup Assistant

Never run `sim.sh` alongside `driver.sh` / `moveit.sh`.

13.3 Python tools

Tool	Purpose	Needs
`teleop_key.py`	keyboard control (§9)	driver or sim
`hover_test.py`	hover over named squares to check board alignment	driver or sim
`place_test.py`	round-trip a piece to measure placement error	driver
`reach_check.py`	reachability map for all 64 squares	nothing
`chess_game.py`	the chess demo	driver or sim, stockfish
`gen_board.py`	generate the printable board with ArUco corners	nothing
`gen_aruco_markers.py`	generate ArUco markers alone	nothing
`calibrate_camera.py`	homography from the chess board (preferred)	camera
`compute_homography.py`	manual 4-point homography	nothing

Tool	Purpose	Needs
`labelme_to_yolo.py`	convert LabelMe labels to YOLO format	nothing
`prepare_yolo_dataset.py`	split images into a YOLO train/val set	nothing
`serial_wiggle.py`	raw-serial "is the hardware alive?" test	serial, **driver stopped**
`gripper_test.py`	raw-serial claw sweep	serial, **driver stopped**
`probe_protocol.py`	hunt for an undocumented servo-read command	serial, **driver stopped**
`arm_client.py`	shared IK + motion library, imported rather than run	none
`board_config.py`	shared board maths, imported rather than run	none

The three raw-serial tools talk to `/dev/ttyUSB0` directly. **Stop the driver first.** Two programs writing one serial port corrupt each other's packets.

13.4 Configuration files

File	Controls
`config/board.yaml`	where the chess board is, and per-square corrections
`dofbot_vision/config/picking.yaml`	homography, heights, gripper, target classes
`dofbot_moveit_config/config/moveit_params.yaml`	planner settings, default speed scaling
`dofbot_moveit_config/config/joint_limits.yaml`	joint speed and acceleration limits
`dofbot_moveit_config/config/kinematics.yaml`	IK solver choice, `position_only_ik`
`dofbot_description/urdf/dofbot.urdf.xacro`	the robot's physical dimensions

13.5 Command cheat sheet

```
# Build
cd dofbot_ros2_ws && colcon build --symlink-install && source install/setup.bash
ros-build                                  # container shortcut

# Bring up (hardware)
scripts/container.sh && scripts/usb.sh    # WSL host
scripts/driver.sh                          # container terminal 1
scripts/moveit.sh                          # container terminal 2
scripts/status.sh                          # container terminal 3

# Bring up (simulation)
scripts/sim.sh

# Look around
ros2 node list
ros2 topic list
ros2 topic echo /joint_states --once
ros2 topic hz /image_raw
ros2 action list

# Drive it
python3 tools/teleop_key.py
python3 tools/hover_test.py --check-only a1 h1 d4
python3 tools/reach_check.py
python3 tools/chess_game.py --skill 3
```

14. Troubleshooting

14.1 Environment and build

Symptom	Cause → fix
`ros2: command not found`	terminal not sourced → `source /opt/ros/humble/setup.bash` (§5.3)
`Package 'dofbot_bringup' not found`	workspace not sourced, or not built → `source install/setup.bash`
`ModuleNotFoundError: serial`	`apt install python3-serial`
`ModuleNotFoundError: chess`	`pip3 install chess`
`_ARRAY_API not found` / `KeyError: 16` / `canonicalize_version()`	a pip pin was bypassed → §2.3.4
Edited a file, nothing changed	you ran `install/`, not `src/` → rebuild, or use `--symlink-install` (§5.5)
Build behaves impossibly	stale artifacts → `rm -rf build install log` and rebuild
Two copies of everything	you built in the repo root → delete the top-level `build/ install/ log/` (§5.6)

14.2 RViz and planning

Symptom	Cause → fix
RViz opens, no robot / `robot_description` errors	stale or unsourced build → clean rebuild
RViz renders garbage	add `-e LIBGL_ALWAYS_SOFTWARE=1` to the container
Controllers stuck `inactive`	plugin load error in `ros2_control_node` output, usually stale artifacts
Dragging the orientation ring does nothing	expected, because IK is position-only (§6.4.1)
Planning always fails	target outside the workspace. Verify with `reach_check.py` (§10.5)
IK error `-31` on every pick	`ros2 param get /move_group robot_description_kinematics.arm.position_only_ik` must be `True`

14.3 The arm

Symptom	Cause → fix
Execute succeeds, arm does not move	driver not running or serial not open → check terminal 1 for `Connected to /dev/ttyUSB0`
No `/dev/ttyUSB*`	WSL: re-run `scripts/usb.sh`. Native: check `dmesg | tail`, cable, `dialout` group

Symptom	Cause → fix
`Permission denied: /dev/ttyUSB0`	not in `dialout` → `sudo usermod -aG dialout $USER`, then log out and in
Arm jumps violently at startup	it was not centered before the driver started → power cycle, press K1, restart (§1.5)
Arm stops responding mid-session	USB dropped → the watcher re-attaches, but repeated drops mean power trouble: check the DC supply and cable
Motion is jerky / pulsing	something is sending dense waypoints. Check `min_segment_ms` is still 3000 (§7.4)
Trajectory goal rejected	another motion is running, or two `move_group` nodes are up → `ros2 node list`
Servo buzzes and gets hot	it is straining against a limit or an obstacle. Power off. It cannot tell you (§7.10)
Arm reaches the wrong place, consistently	open-loop offset. It did not start centered, or `tool_len` needs adjusting

14.4 Network (Pi setup)

See §4.7.

14.5 Camera and vision

Symptom	Cause → fix
Camera attaches in WSL but `/image_raw` is silent; `dmesg` spams `vhci ... Not yet implemented`	usbipd cannot stream webcams → use the ffmpeg bridge (§11.2.2)
Bridge node: `Connection refused`	ffmpeg not running (it exits when a client disconnects) → restart `stream_camera.ps1` and check the firewall allows port 8090
Blurry image	manual focus → twist the lens barrel
No detections at all	object not in YOLO's 80 classes, or `min_confidence` too high, or too dark
Detections fine, picks are offset	camera moved after calibration → recalibrate (§11.4)
Picks offset by a constant amount	table height changed, or the object is taller than the calibration plane (§11.1)
Chessboard calibration finds no corners	board not empty, uneven lighting, or partly out of frame

15. Glossary

Action. A ROS request that takes time, reports progress, and can be cancelled. Used for "move the arm", where a plain message would not tell you when it finished.

colcon. The build tool for ROS 2 workspaces (§5.5).

DDS. The networking layer underneath ROS 2. It is what makes nodes on different machines find each other automatically (§4.1).

Forward kinematics (FK). Given the joint angles, where is the gripper? Easy, one calculation.

Homography. The 3×3 matrix that converts camera pixels to table coordinates, valid because everything is on one flat plane (§11.1).

Inverse kinematics (IK). Given a target position, what should the joint angles be? Hard, because there may be several answers or none (§7.6).

Joint. One motor's axis of rotation. This arm has 5 plus a gripper.

Launch file. A Python script that starts several nodes at once with the right settings. `ros2 launch <package> <file>`.

Link. A rigid part of the robot, between joints.

MoveIt. The motion planning framework. It works out a path from A to B.

Node. One running program in ROS. Nodes talk over topics, services, and actions.

Open loop. Commanding without feedback. This arm cannot report where it really is, so everything is open loop (§7.10).

Package. One folder of related code with a `package.xml` (§5.4).

Pose. A position plus an orientation.

RViz. The 3D visualizer. It draws what ROS believes is happening and does not itself control anything.

Servo. A motor that holds a commanded angle. Six of them here.

SRDF. A companion to the URDF that groups joints (`arm`, `gripper`) for MoveIt.

Topic. A named stream of messages such as `/joint_states`. Publishers send and subscribers receive, and neither knows the other exists.

Trajectory. A list of joint positions with timestamps (§7.3).

URDF. The XML file describing the robot's physical structure (§7.2).

Workspace. The folder holding your ROS packages and their build output (§5.1).

YOLO. The neural network used for object detection (§11.3).